

企业财务数据获取

- **爬虫：**按照一定规则，自动抓取网络信息的程序
- **爬虫基本流程：**发送请求→获取响应内容→解析内容→保存数据
- **Requests库：**发送网络请求，返回响应数据
- **科云第三方库爬虫函数：** spider
- **Tushare：**是基于Python/R语言的财经数据接口库，数据内容将扩大到包含股票、基金、期货、债券、外汇、行业大数据
- **数据获取：**初始化pro接口: `pro_api('your token')`

内容概要

爬虫：按照一定规则，自动抓取网络信息的程序

爬虫基本流程：发送请求→获取响应内容→解析内容→保存数据

Requests库：发送网络请求，返回响应数据

自定义爬虫函数：spider

Tushare：是基于Python/R语言的财经数据接口库，数据内容将扩大到包含股票、基金、期货、债券、外汇、行业大数据

数据获取：初始化pro接口：pro_api('your token')

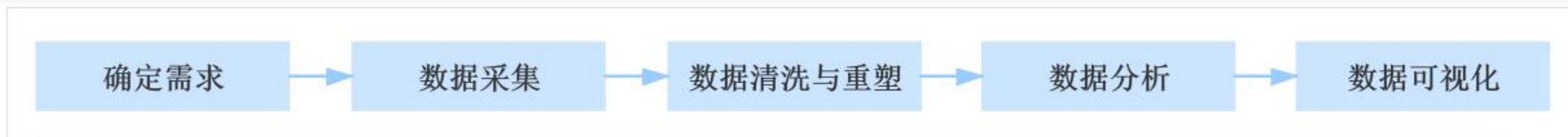
目录

1 了解爬虫工作原理

2 运用爬虫工具获取数据

爬虫工作原理

大数据分析是基于商业目的，有目标的进行数据收集、处理、加工、分析，提炼出有价值的信息的过程。



数据及获取方式：

企业产生的数据：在企业生产运营中会产生与自身业务相关的大量数据。

数据平台购买的数据：数据平台是以数据交易为主营业务的平台。

数据管理咨询公司的数据：数据管理咨询公司为提供专业的咨询服务，会收集大量与特定业务相关的数据作为支撑。

政府、机构公开的数据：政府会发布一些公开的统计数据或信息，成为行业内权威信息的来源。

爬取的网络数据：利用爬虫技术，即可自动抓取所需要的数据，获取更多数据源，提高数据分析的效率和效果。

爬虫的概念

爬虫(即网络爬虫的简称), 是一种按照一定规则, 自动抓取网络信息的程序。

爬虫可以理解为在网络上爬行的一只蜘蛛, 互联网就像一张大网, 爬虫便是在这张网上爬来爬去的蜘蛛, 如果遇到猎物(即所需的资源), 就会将其抓取下来。所以爬虫的目的在于将目标网页数据下载至本地, 以便于进行后续的数据分析。

```
# 爬取该行业上市公司财务报表摘要
for i in codeList:
    spider('https://keyun-oss.oss-cn-beijing.aliyuncs.com/app/bigdata/2019/company/year/cwbbzy_'+i+'.csv', './行业数据/'+i+'_cwbbzy.csv')
```

```
文件: ./行业数据/600777_cwbbzy.csv 下载成功!
文件: ./行业数据/000552_cwbbzy.csv 下载成功!
文件: ./行业数据/600121_cwbbzy.csv 下载成功!
文件: ./行业数据/600403_cwbbzy.csv 下载成功!
文件: ./行业数据/600871_cwbbzy.csv 下载成功!
文件: ./行业数据/300164_cwbbzy.csv 下载成功!
文件: ./行业数据/600759_cwbbzy.csv 下载成功!
文件: ./行业数据/300084_cwbbzy.csv 下载成功!
文件: ./行业数据/000506_cwbbzy.csv 下载成功!
文件: ./行业数据/601225_cwbbzy.csv 下载成功!
文件: ./行业数据/600711_cwbbzy.csv 下载成功!
文件: ./行业数据/002554_cwbbzy.csv 下载成功!
```

Python是一门非常适合网络爬虫的编程语言, 它能提供许多爬虫相关的库(如requests库), 可以高效实现网页抓取, 并可用极短的代码完成网页的标签过滤功能。

浏览网页的过程

浏览网页的过程可以总结为以下几步：

第1步，用户输入网址，计算机提取域名。

第2步，浏览器查找域名对应的IP地址。

第3步，浏览器获取IP地址后，向此IP地址发起对该资源的访问请求。

第4步，服务端响应请求，并把相应的数据传给浏览器(返回html页面)，浏览器将html页面解析后就是我们看到的文字和图片。

HTML(超文本标记语言)是用来描述网页的一种语言。用户看到的网页实质是由HTML代码构成的。

爬虫原理

爬虫就是模拟用户浏览网页的操作，通过URL模拟浏览器向网站发送请求，获取资源后提取有用的数据并保存。所以，原则上只要浏览器能做的事情，爬虫都能做到。

从理论上讲，网络上的资源都可以获取，**爬取数据类型包括HTML文档、json格式化文本、二进制文件(图片和视频)**以及其他各类数据。

json是一种轻量级的数据交换格式，易于编写和阅读，也易于机器解析，是理想的数据交换语言。json文本格式类似于Python中的字典，在爬虫中使用非常广泛。

爬虫基本流程



发送请求：通过url向服务器发送HTTP请求。

获取响应内容：若服务器正常响应，会返回一个Response响应(即所要获取的页面内容，可能为html、json、二进制数据等)。

解析内容：对返回的响应内容进行解析，提取所需数据。

保存数据：可将数据保存为各种形式，如数据库或特定格式的文件(如：json、csv文件等)。

HTTP协议与URL



HTTP协议即超文本传输协议，是互联网上应用最为广泛的一种网络协议，所有网页文件都必须遵守这个标准，设计HTTP最初的目的就是提供一种发布和接收HTML页面的方法。

URL是统一资源定位符，也就是网址。URL是对互联网上资源位置和访问方法的一种简洁的表示，是互联网上标准资源的地址。**互联网上的每个文件都有一个唯一的URL**，它包含的信息指出文件的位置以及浏览器应该怎么处理它。



02

运用爬虫工具获取数据

HTTP协议对资源的操作



方法	说明
GET	请求获取URL位置的资源
HEAD	请求获取URL位置资源的响应消息报告，即获得资源的头部信息
POST	请求向URL位置的资源后附加新的消息
PUT	请求向URL位置存储一个资源，覆盖原URL位置的资源
PATCH	请求局部更新URL位置的资源,即改变该处资源的部分内容
DELETE	请求删除URL位置存储的资源

HTTP协议函数

Requests简介

Requests库是一个功能强大的网络请求库，可以请求网站获取网页上的数据。

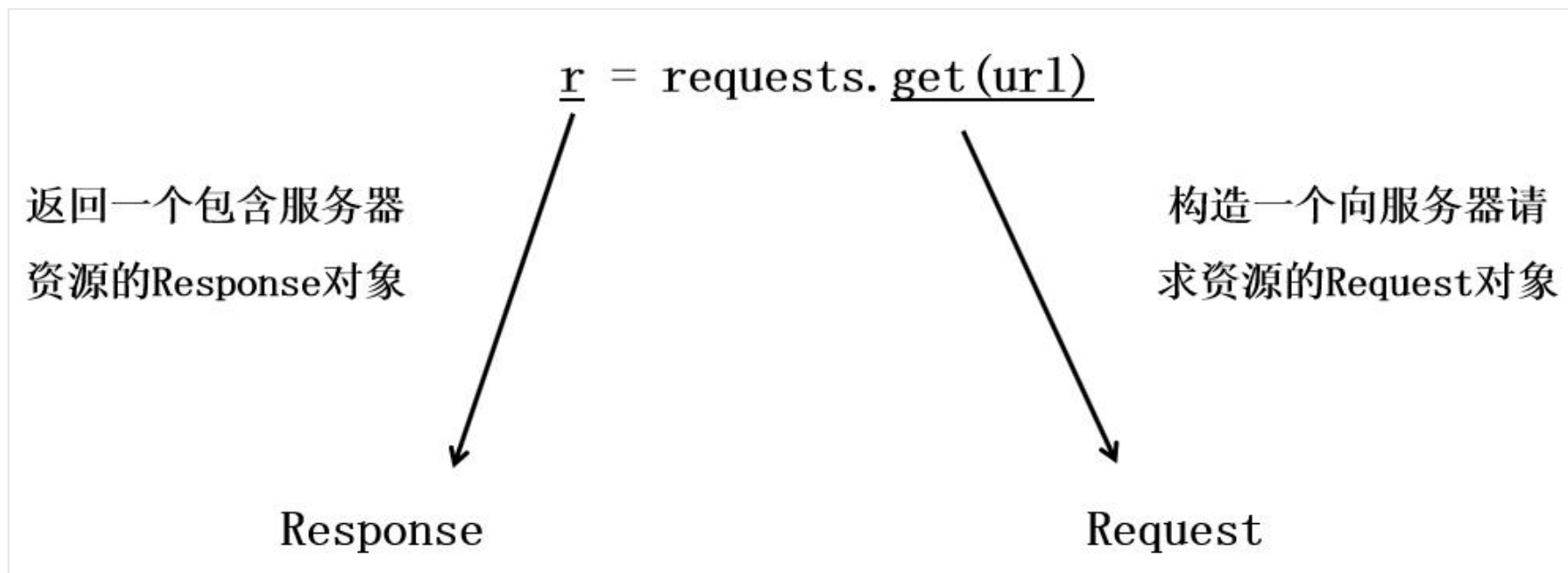
Requests引入规则为：import requests

方法	说明
requests.request()	构造一个请求，最基本的方法，是下面方法的支撑
requests.get()	获取html网页
requests.head()	获取html网页的头信息
requests.post()	向html网页提交追加资源请求
requests.put()	向html网页提交覆盖资源请求
requests.patch()	向html网页提交局部修改请求
requests.delete()	向html网页提交删除请求

requests.get()

爬取网页最简单的方法为：`r = requests.get(url)`

通过requests调用get()方法，传入需要获取资源的url，即可构造一个HTTP请求，并获取响应内容。



Response简介

Response对象包含爬虫返回的内容

属性	说明
r.status_code	请求返回状态码, 200表示连接成功 , 404或其他表示失败
r.text	响应内容的字符串形式, 即url对应的页面内容
r.content	响应内容的二进制形式
r.headers	网页头部信息, 有关响应的补充信息, 如响应数据的文件类型等
r.encoding	从网页头信息(header)中猜测的响应内容编码方式
r.apparent_encoding	从内容中分析出的响应内容编码方式(备选编码方式)

注意: 只有返回状态码为200(即连接成功), 才能正常查看其他属性。

查看r.text

例2-2-1：爬取“科云大数据中心”-平安银行股份有限公司的资产负债表

```
# 引入requests
import requests
```

发送请求(爬取 “科云大数据中心” -平安银行股份有限公司的资产负债表)

```
r = requests.get('https://keyun-oss.acctedu.com/app/bigdata/2019/company/year/zcfzb 000001.csv')
```

```
# 检测请求的状态码
print(r.status_code)
```

```
#查看响应内容
r.text
```



200

[illegible]

运行结果

编码问题

请求发出后，requests会基于HTTP头部信息对响应的编码方式做出推测，当访问`r.text`时，requests会使用其推测的编码方式进行解码，并显示网页内容。

查看从头信息猜测出的编码方式：

`r.encoding`

运行结果：'ISO-8859-1'

查看响应内容的真实编码方式：

`r.apparent_encoding`

运行结果：'utf-8'

requests中编码函数

属性	说明
<code>r.encoding</code>	从网页头信息(header)中猜测的响应内容编码方式
<code>r.apparent_encoding</code>	从内容中分析出的响应内容编码方式(备选编码方式)

解决以上乱码解决方法：`r.encoding = r.apparent_encoding`
通过`r.apparent_encoding`属性获取响应内容真实的编码方法，再用`r.encoding`指定该编码，`r.text`将会根据指定的编码解析网页。

编码问题

例2-2-2：指定真实编码，并查看响应内容

```
r.encoding = r.apparent_encoding # 指定编码
```

```
r.text # 查看响应内容
```

[illegible]

改变编码后，访问r.text时，requests都将使用r.encoding的新值，这样就可以返回正常的文本内容了。

Response对象异常

在访问网页的过程中，常常会遇到许多异常情况，如：网络连接错误、请求超时等等，一旦出现异常，就无法获取响应内容。

获取网页时，只有Response对象返回状态码为200(即连接成功)，才能顺利爬取网页内容，`r.status_code`属性返回状态码，那么，爬虫过程中可以使用if语句进行判断

```
if r.status_code == 200:
```

代码块1

```
else:
```

代码块2



`r.raise_for_status()`: 在方法内部判断`r.status_code`是否为200，如果不是200，则产生异常。使用该方法就不需要增加if语句，可以简化代码

数据保存

使用`requests.get()`爬取数据后，需要将数据保存至本地，方便后续直接调用，保存文件类型包括json格式、csv格式、Excel格式等等，具体保存为什么类型的文件可以根据爬取到的文件类型确定。

with as 语句基本语法格式：

with 表达式 as 变量: # 表示将表达式返回的结果保存到该变量中
代码块

保存数据方法：

with open('文件名称','写入模式',编码方式) as f:
f.write()

open()函数

open()函数：可以打开一个文件，并创建一个file对象，函数语法如下：
open(file, mode='r', buffering=-1, encoding=None, errors=None, newline=None, closefd=True, opener=None)

常用参数	说明
file	打开并返回一个文件对象
mode	打开文件的模式，默认为“r”，表示只读
encoding	设定打开文件时使用的编码方式

file对象常用参数

r、w、a表示处理文本文件，需要指定编码方式，一般设置encoding='utf-8'，rb、wb、ab表示处理二进制文件，不需要指定编码方式。

模式	说明
r	以只读方式打开文件，默认模式，文件必须存在
w	打开一个文件用于重新写入，若该文件已经存在则打开文件，并从开头开始编辑，原有内容被覆盖，若该文件不存在，则创建新文件写入。
a	打开一个文件用于追加，若文件存在，新的内容将追加写入已有内容之后，若该文件不存在，则创建新文件写入。
b	b 表示处理二进制文件，包括 rb、wb、ab

文件打开几种模式

write()函数

write()函数：表示向文件中写入指定内容，其语法：write(data)

data：表示写入的数据，包括是字符串类型(str)、二进制类型(bytes)。

文件打开模式带b(即rb、wb、ab)时，写入的数据必须是二进制类型。

字符串与二进制类型是否可以转换：

str通过encode方法转化为bytes

bytes通过decode方法转化为str

注意：编码方式与解码方式必须一样，否则将产生乱码；此外，默认编码的方式为：utf-8。

#字符串转换为二进制

```
a='1234'
```

```
b=a.encode()
```

```
print(b)
```

```
print(type(b))
```

思考：如何把二进制转成字符串？

数据写入形式

write()函数可以写入字符串形式，也可以写入二进制形式，那么，在保存数据时到底该使用什么形式写入呢？

常用文件类型：

r.text: json文件、csv文件等文本型文件。

r.content: Excel文件(xlsx/xls)、图片类文件(png/jpg)、视频类文件(mp4)等。

```
# r.text写入文件：写入文本文件必须指定编码方式
with open('文件名称','w',encoding='utf-8') as f:
    f.write(r.text)
```

```
# r.content写入文件，无需指定编码方式
with open('文件名称','wb') as f:
    f.write(r.content)
```

数据写入形式

例2-2-3：在例2-2-2中，爬取到的资产负债表是以csv文件存储的，则直接将r.text写入一个csv文件即可保存。

```
# r.text写入文件，文件类型为csv
with open('000001-zcfzb.csv','w',encoding='utf-8') as f:
    f.write(r.text)
```

运行结果



jupyterhub000001-zcfzb.csv

LogoutControl Panel

FileEditViewLanguagecurrent mode

1	报告日期, 2019-12-31, 2018-12-31, 2017-12-31, 2016-12-31, 2015-12-31, 2014-12-31, 2013-12-31, 2012-12-31, 2011-12-31, 2010-12-31, 2009-12-31, 2008-12-31, 2007-12-31, 2006-12-31, 2005-12-31, 2004-12-31,
2	营业总收入(万元), 13795800, 11671600, 10578600, 10771500, 9616300, 7340700, 5218900, 3974865, 2964306, 1802228, 1511444, 1451312, 1080750, 781787, 545055, 531012,
3	营业收入(万元), --, --, --, --, --, --, --, --, 2964306, 1802228, 1511444, 1451312, 1080750, 781787, 545055, 531012,
4	利息收入(万元), 17754900, 16288800, 14806800, 13111900, 13164900, 11920200, 9310200, 7461368, 5233070, 2625179, 2198551, 2646526, 1804390, 1206842, 866401, 865725,
5	已赚保费(万元), --, --, --, --, --, --, --, --, --, --, --, --, --, --, --, --,
6	手续费及佣金收入(万元), 4590300, 3936200, 3572500, 3130900, 2918500, 1970600, 1182100, 644952, 412960, 183638, 138697, 105665, 66775, 42199, 30709, 24745,
7	房地产销售收入(万元), --, --, --, --, --, --, --, --, --, --, --, --, --, --, --, --,
8	其他业务收入(万元), 11000, 17000, 18600, 14600, 16100, 21300, 15000, 15477, 13288, 14750, 12871, 11394, 16497, 11305, 7293, 7575,
9	营业总成本(万元), 4214200, 3654000, 7556300, 7793600, 6726800, 4716100, 3223400, 2220699, 1651036, 1017272, 895531, 1370969, 503180, 379062, 301373, 286987,
10	营业成本(万元), --, --, --, --, --, --, --, --, 1651036, 1017272, 895531, 1370969, 503180, 379062, 301373, 286987,
11	利息支出(万元), 8758800, 8814300, 7405900, 5470800, 6555000, 6615600, 5241400, 4157832, 2704092, 1042260, 900114, 1386738, 843805, 506877, 377034, 372395,
12	手续费及佣金支出(万元), 916000, 806500, 505100, 345000, 274000, 232800, 136500, 72824, 46493, 25123, 20619, 20526, 14704, 7604, 6799, 5964,
13	房地产销售成本(万元), --, --, --, --, --, --, --, --, --, --, --, --, --, --, --, --,
14	研发费用(万元), --, --, --, --, --, --, --, --, --, --, --, --, --, --, --, --,
15	退保金(万元), --, --, --, --, --, --, --, --, --, --, --, --, --, --, --, --,
16	赔付支出净额(万元), --, --, --, --, --, --, --, --, --, --, --, --, --, --, --, --,
17	提取保险合同准备金净额(万元), --, --, --, --, --, --, --, --, --, --, --, --, --, --, --, --,
18	保单红利支出(万元), --, --, --, --, --, --, --, --, --, --, --, --, --, --, --, --,
19	分保费用(万元), --, --, --, --, --, --, --, --, --, --, --, --, --, --, --, --,
20	其他业务成本(万元), --, --, --, --, --, --, --, --, --, --, --, --, --, --, --, --,
21	营业税金及附加(万元), 129000, 114900, 102200, 344500, 667100, 548200, 406500, 341199, 250634, 132460, 106913, 115167, 82431, 55347, 41692, 41257,
22	销售费用(万元), 4085200, 3539100, 3161600, 2797300, 3011200, 2666800, 2127900, 1566439, 1185544, 736001, 631109, 522387, 420749, 323715, 259681, 245730,
23	管理费用(万元), --, --, --, --, --, --, --, --, --, --, --, --, --, --, --, --,
24	财务费用(万元), --, --, --, --, --, --, --, --, --, --, --, --, --, --, --, --,
25	资产减值损失(万元), --, --, --, 4292500, 4651800, 3048500, 1501100, 689000, 313061, 214857, 148812, 157509, 733416, 205376, 198622, 180278, 188985,
26	公允价值变动收益(万元), 4900, 89200, -6100, 4900, 10700, -1000, 71100, -2922, 601, 1800, -133, 6580, 8217, 11361, 112, -988,

try except语句

try except语句：Python中的异常处理语句，可以捕获并处理异常。

try except语句语法

```
try:
    代码块1 # 可能产生异常的代码
except:
    代码块2 # 异常处理代码
```

例2-2-4：输入一个数字，如果不是数字则重新输入

while True:

try:

```
x = int(input("请输入一个数字: "))
```

```
break
```

except:

```
print("您输入的不是数字，请再次尝试输入！")
```

运行结果:

请输入一个数字: a

您输入的不是数字，请再次尝试输入！

请输入一个数字: s

您输入的不是数字，请再次尝试输入！

请输入一个数字: 2

自定义爬虫函数

例2-2-5：根据Requests库相关内容自定义爬虫函数(仅考虑Excel、json、csv三种文件类型)。

```
# 自定义爬虫函数：url、filename均为字符串类型
import requests # 引入库
def spider(url,filename):
    try:
        r = requests.get(url,timeout=5)
        r.raise_for_status()
        r.encoding = r.apparent_encoding
        if(filename.endswith(".xls") or filename.endswith(".xlsx")):
            with open(filename,'wb') as f:
                f.write(r.content)
        else:
            with open(filename,'w',encoding='utf-8') as f:
                f.write(r.text)
        return print('文件：',filename,'下载成功！ ')
    except:
        return print('下载失败！')
```

timeout： 设定超时时间，以秒为单位，为防止服务器响应缓慢，可以设置该参数，若在指定时间内没有收到响应，则抛出异常。

str.endswith()： 判断字符串是否以某字符结束。

自定义爬虫函数

例2-2-6：登录“科云大数据中心”，使用自定义爬虫函数爬取“采矿业”所有上市公司股票代码及相关信息，并将爬取数据保存为json文件，文件名为“cky.json”。

```
spider('https://keyun-oss.acctedu.com/app/bigdata/2019/block/hy002000.json','cky.json')
```

运行结果:

文件：cky.json 下载成功!



财务案例实践

根据requests库相关内容自定义爬虫函数:

- (1)使用try except语句, try语句如果正常执行, 则返回print('文件: ',filename,'下载成功! '); 如果try语句执行发生异常情况, 则执行except语句返回print('下载失败! ')。
- (2)爬取的数据仅考虑json、 csv两种文件类型,写入模式为"w"。
- (3)使用自定义函数爬取万科企业股份有限公司(000002)公司简介, 文件保存为json格式, 文件名为:000002.json。
(注: url='https://keyun-oss.acctedu.com/app/bigdata/2019/company/info/000002.json')

```
# 引入requests
import requests
# 自定义爬虫函数
def proc(url,filename):
    ####修改代码开始####
    try:

        return print('文件: ',filename,'下载成功! ')
    except:
        return print('下载失败! ')
    ####修改代码结束####
# 调用函数爬取万科企业股份有限公司简介
proc('https://keyun-oss.acctedu.com/app/bigdata/2019/company/info/000002.json','000002.json')
```

参考答案:
用于清除字符串中的 “#”及 “,”代码

```
try:
    r = requests.get(url)
    r.raise_for_status()
    r.encoding = r.apparent_encoding
    with open(filename,'w',encoding='utf-8') as f:
        f.write(r.text)
    return print('文件: ',filename,'下载成功! ')
except:
    return print('下载失败! ')
```

金融数据接口

目录

01

数据接口简介

02

Tushare

03

数据获取

数据接口：是软件或系统间为了交换数据而预先定义的连接点或通道，它规定了数据如何被格式化、请求、传输和响应。

Tushare
付费

AKShare
免费

baostock
免费

提示

以上金融数据接口官网均提供文档支持，每个数据接口均提供详细的说明和示例，用户只需复制粘贴就可以下载数据。

数据接口的作用

数据接口提供了更方便的方式来收集所需的数据，无论是**实时数据**还是**历史数据**，只需要连接到大型数据库或数据仓库，就可以获取**更丰富、更全面**的数据集，这对于数据分析至关重要，它不仅提供了**分析的基础**，还提高了分析的**效率和质量**，使得数据分析**更加灵活、可靠和深入**。

Tushare是基于Python/R 语言的财经数据接口库。主要实现对股票等金融数据从数据采集、清洗加工到数据存储的过程，Tushare Pro版做了更大的改进，数据内容将扩大到包含股票、基金、期货、债券、外汇、行业大数据，同时包括了数字货币行情等区块链数据的全数据品类的金融大数据平台，为各类金融投资和研究人员提供适用的数据和工具。

- 量化投资分析师（Quant）
- 对金融市场进行大数据分析的企业和个人
- 开发以证券为基础的金融类产品和解决方案的公司
- 正在学习利用python进行数据分析的人

【注：Tushare不是普通炒股者用的软件，而是为那些有兴趣做股票期货数据分析的人提供pandas矩阵数据的工具】

下载安装方式

- 方式1:

```
pip install tushare
```

如果安装网络超时可尝试国内pip源, 如

```
pip install tushare -i https://pypi.tuna.tsinghua.edu.cn/simple
```

- 方式2: 访问<https://pypi.python.org/pypi/tushare/>下载安装, 执行

```
python setup.py install
```

- 方式3: 访问<https://github.com/waditu/tushare>, 将项目下载或者clone到本地, 进入到项目的目录下, 执行: `python setup.py install`

使用步骤

1.Tushare用户注册

访问Tushare社区门户（<https://tushare.pro>），点击右上角“注册”

2.获取`token`

在“用户中心”中点击“接口`TOKEN`”

3.获取数据

注意：`token`是调取数据的唯一凭证，请妥善保管，如发现别人盗用，可在本页面点击“刷新”操作之前的。

设置token这个步骤只需要在第一次或者token失效后调用，完成调取tushare数据凭证的设置，正常情况下不需要重复设置，或不想保存token到本地，也可以忽略此步骤，在初始化pro接口里直接设置token：
`pro_api('your token')`完成初始化。

代码

```
# 导入tushare
import tushare as ts

# 设置token
ts.set_token('your token here')

# 初始化pro接口
pro = ts.pro_api('your token')
```

在Tushare数据接口里，股票代码参数都叫ts_code，每种股票代码都有规范的后缀。

交易所名称	交易所代码	股票代码后缀	备注
上海证券交易所	SSE	.SH	600000.SH(股票) 000001.SH(指数)
深圳证券交易所	SZSE	.SZ	000001.SZ(股票) 399005.SZ(指数)
北京证券交易所	BSE	.BJ	8和4开头的股票
香港证券交易所	HKEX	.HK	00001.HK



沪深股票

基础数据

行情数据

日线行情

周线行情

月线行情

股票周/月线行情(每日更新)

复权行情

复权因子

实时行情(爬虫)

实时成交(爬虫)

实时排名(爬虫)

每日行情

Search



A股日线行情

单击“数据工具”

接口: daily, 可以通过[数据工具](#)调试和查看数据

数据说明: 交易日每天15点~16点之间入库。本接口是未复权行情, 停牌期间不提供数据

调取说明: 120积分每分钟内最多调取500次, 每次6000条数据, 相当于单次提取23年历史

描述: 获取股票行情数据, 或通过[通用行情接口](#)获取数据, 包含了前后复权数据

输入参数

名称	类型	必选	描述
ts_code	str	N	股票代码 (支持多个股票同时提取, 逗号分隔)
trade_date	str	N	交易日期 (YYYYMMDD)

数据接口

沪深股票

基础数据

行情数据

日线行情

周线行情

股票周/月线行情
(每日更新)

月线行情

复权行情

复权因子

实时行情 (爬虫)

实时成交 (爬虫)

实时排名 (爬虫)

每日指标

通用行情接口

每日涨跌停价格

每日停复牌信息

沪深股通十大成交
股

港股通十大成交股

入参 请输入参数值

ts_code 000858.SZ 股票代码

trade_date 20241217 交易日期

start_date 开始日期 开始日期

end_date 结束日期 结束日期

offset 开始行数 开始行数

limit 最大行数 最大行数

运行调试

生成代码

详细文档

数据表

返回 请勾选需要显示的列

☐	字段	类型	说明
☒	ts_code	str	股票代码
☒	trade_date	str	交易日期
☐	open	float	开盘价
☐	high	float	最高价
☐	low	float	最低价
☒	close	float	收盘价
☐	pre_close	float	昨收价
☒	change	float	涨跌额
☒	pct_chg	float	涨跌幅
☐	vol	float	成交量
☒	amount	float	成交额

导出CSV

数据获取

代码

```
# 导入tushare
import tushare as ts
# 初始化pro接口
pro = ts.pro_api('your token')

# 拉取数据
df = pro.daily(**{
    "ts_code": "000858.SZ",
    "trade_date": 20241217,
    "start_date": "",
    "end_date": "",
    "offset": "",
    "limit": ""},
    fields=[
        "ts_code",
        "trade_date",
        "close",
        "change",
        "pct_chg",
        "amount"])
show_table(df)
```


功能特点

- **数据丰富**: 覆盖面广, 数据种类多样。
- **更新及时**: 实时数据和历史数据更新迅速。
- **易用性强**: API接口简洁, 易于理解和使用。
- **灵活性高**: 支持灵活的数据查询和自定义数据请求。

- **数据接口的作用**: 在数据获取方面极大地**减轻工作量**
- **Tushare**: 是基于Python/R语言的财经数据接口库, 数据内容将扩大到包含股票、基金、期货、债券、外汇、行业大数据
- **导入tushare**: `import tushare as ts`
- **数据获取**: 正常情况下可以在初始化pro接口里直接设置token:
`pro_api('your token')`完成初始化